

Higgsfield — Revenue Drivers and Retention

Revenue has four drivers — acquisition, retention, expansion, recovery — under a margin constraint.

This notebook:

1. **§1 Revenue drivers** — decompose six months of revenue into the four types, and check how the mix moves over time.
2. **§2 Retention** — choose the retention window from the data, then read survival by period and by cohort.

Companion notebook: `higgsfield_drivers_2_expansion_segments.ipynb`
(expansion, journey, hypothesis tests, segments).

0. Setup

```
In [1]: import pandas as pd
import numpy as np
import nbformat # required for plotly fig.show() mime rendering in notebook
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import plotly.io._renderers as _plotly_renderers
from _plotly_utils.optional_imports import _not_importable

# Plotly caches failed optional imports for the kernel lifetime; clear + reload
_not_importable.discard('nbformat')
_plotly_renderers.nbformat = nbformat

pd.set_option('display.max_columns', 60)
pd.set_option('display.width', 220)
pd.set_option('display.float_format', '{:,.2f}'.format)

PALETTE = ['#6C5CE7', '#00B894', '#FDCB6E', '#E17055', '#0984E3', '#2D3436',
px.defaults.template = 'plotly_white'
px.defaults.color_discrete_sequence = PALETTE
px.defaults.height = 420

PATH = 'data/' # folder with the csv files
```

```
In [2]: generations_df = pd.read_csv(PATH + 'generations.csv')
customers_df = pd.read_csv(PATH + 'customers.csv')
charges_df = pd.read_csv(PATH + 'charges.csv')

def to_utc(s):
    return pd.to_datetime(s.astype(str).str.removesuffix(' UTC'), format='%Y-%m-%d %H:%M:%S')
```

```

for c in ['in_progress_at', 'completed_at', 'failed_at']:
    generations_df[c] = to_utc(generations_df[c])
customers_df['created_at'] = to_utc(customers_df['created_at'])
charges_df['revenue_time'] = to_utc(charges_df['revenue_time'])

OUTLIERS = list(generations_df['user_id'].value_counts().index[:1])
drop_cust = set(customers_df.loc[customers_df['user_id'].isin(OUTLIERS), 'customer_id'])
generations_df = generations_df[~generations_df['user_id'].isin(OUTLIERS)].copy()
customers_df = customers_df[~customers_df['user_id'].isin(OUTLIERS)].copy()
charges_df = charges_df[~charges_df['customer_id'].isin(drop_cust)].copy()

generations_df['failed'] = generations_df['failed_at'].notna() & generations_df['completed_at'].isna()
generations_df['is_video'] = (generations_df['type'] == 'video').astype(int)
charges_df['is_credits'] = charges_df['subscription_plan'] == 'Credits Package'

DATA_END = max(generations_df['in_progress_at'].max(), charges_df['revenue_time'].max())
DATA_END_BILLING = charges_df['revenue_time'].max()

print(f'generations {len(generations_df):,} | customers {len(customers_df):,} | charges {len(charges_df):,}')
print(f'data ends {DATA_END} | last charge {DATA_END_BILLING} | gap {(DATA_END - DATA_END_BILLING).total_seconds():,}')

```

generations 4,949,426 | customers 19,534 | charges 47,291
data ends 2025-10-01 00:13:14.998436+00:00 | last charge 2025-09-30 23:53:39+00:00 | gap 0d

1. Revenue drivers

Every charge is classified into one of four types. The classification is behavioural, reconstructed from each customer's payment sequence — `payment_type` alone doesn't distinguish a first payment from a renewal in every case.

type	rule
new	the customer's first charge
expansion	an upgrade, or a credits purchase by an existing subscriber
recovery	a reactivation, or a payment after a gap longer than the billing period
retained	everything else — a normal renewal

```

In [3]: ch = charges_df.sort_values(['customer', 'revenue_time']).copy()
ch['seq'] = ch.groupby('customer').cumcount()
ch['prev_time'] = ch.groupby('customer')['revenue_time'].shift()
ch['gap_days'] = (ch['revenue_time'] - ch['prev_time']).dt.total_seconds() / 86400

rank = {'Higgsfield Basic': 1, 'Higgsfield Pro': 2, 'Higgsfield Ultimate': 3}
sub_only = ch[~ch['is_credits']].copy()
sub_only['prev_plan'] = sub_only.groupby('customer')['subscription_plan'].shift()
plan_dir = sub_only.set_index('charge_id').apply(
    lambda r: np.sign(rank.get(r['subscription_plan'], 0) - rank.get(r['prev_subscription_plan'], 0))
    if pd.notna(r['prev_plan']) else 0, axis=1)

```

```

ch['plan_dir'] = ch['charge_id'].map(plan_dir).fillna(0)

period_days = ch['subscription_period'].map({'month': 30, 'year': 365})
ch['rev_type'] = np.select(
    [ch['seq'] == 0,
     ch['is_credits'] | (ch['plan_dir'] > 0),
     (ch['payment_type'] == 'Reactivation') | (ch['gap_days'] > period_days),
     ['new', 'expansion', 'recovery'], default='retained')

print(f'{ch["rev_type"].value_counts()}')
print(f'\nrevenue by type:\n{ch.groupby("rev_type")["sales_amount"].agg(["count", "sum", "mean"])}')
share = ch.groupby('rev_type')['sales_amount'].sum() / ch['sales_amount'].sum()
print(f'\nshare of total revenue:\n{share.round(4)}')

```

```

rev_type
new          19532
retained     16390
expansion    10071
recovery      1298
Name: count, dtype: int64

```

```

revenue by type:
              count          sum  mean
rev_type
expansion    10071   405,294.89  40.24
new          19532  1,074,438.97  55.01
recovery      1298    65,477.90  50.45
retained     16390   587,999.93  35.88

```

```

share of total revenue:
rev_type
expansion    0.19
new          0.50
recovery     0.03
retained     0.28
Name: sales_amount, dtype: float64

```

```

In [4]: ch['ym'] = ch['revenue_time'].dt.tz_localize(None).dt.to_period('M').astype('M')
by_m = ch.groupby(['ym', 'rev_type'])['sales_amount'].sum().rename('revenue')
by_m['share'] = by_m.groupby('ym')['revenue'].transform(lambda s: s / s.sum())

fig = px.bar(by_m, x='ym', y='revenue', color='rev_type', text='share',
             title='monthly revenue by driver')
fig.update_traces(texttemplate='%{text:.0%}', textposition='inside')
fig.update_layout(uniformtext_minsize=9, uniformtext_mode='hide')
fig.show()

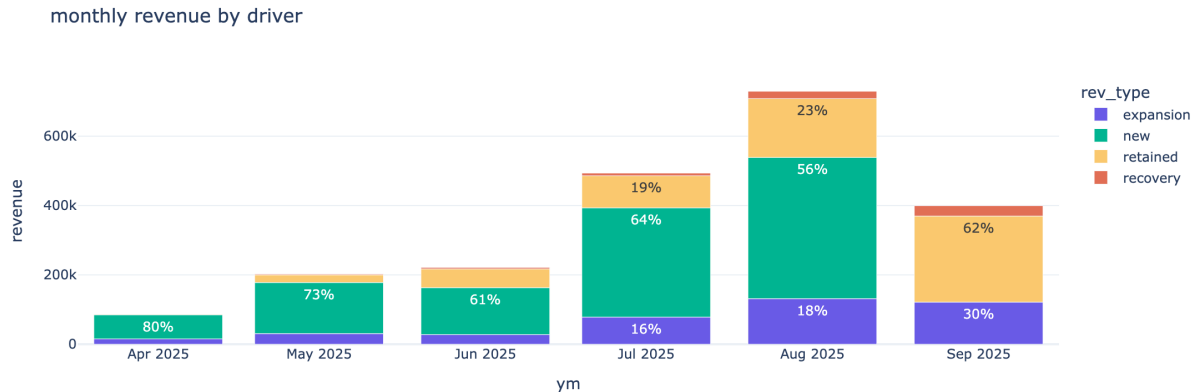
pivot = by_m.pivot(index='ym', columns='rev_type', values='revenue').fillna(0)

# a month with a handful of charges produces a meaningless 100%-of-something
# require at least 2% of the busiest month's revenue before plotting a mix.
vol = pivot.sum(axis=1)
keep = vol >= 0.02 * vol.max()
print(f'months dropped as too thin: {list(vol.index[~keep])}')
pivot = pivot[keep]

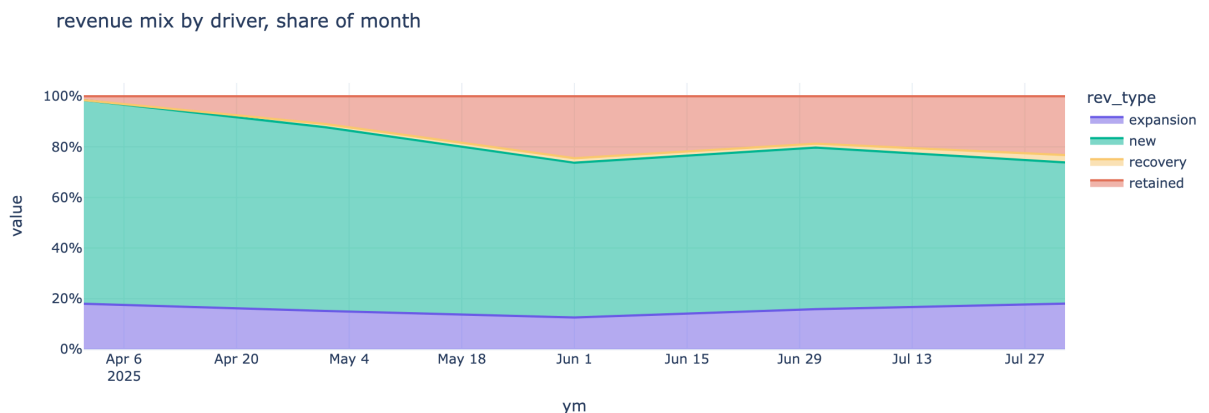
```

```
# last month is incomplete – kept in the data for retention windows, not for
print(f'dropping last month from mix: {pivot.index[-1]}')
pivot = pivot.iloc[:-1]
pct = pivot.div(pivot.sum(axis=1), axis=0)
print(pct.round(3))

fig = px.area(pct, title='revenue mix by driver, share of month')
fig.update_yaxes(tickformat='%.0%')
fig.show()
```



```
months dropped as too thin: []
dropping last month from mix: 2025-09
rev_type expansion new recovery retained
ym
2025-04      0.18 0.81      0.00      0.01
2025-05      0.15 0.73      0.01      0.11
2025-06      0.13 0.61      0.02      0.24
2025-07      0.16 0.64      0.02      0.19
2025-08      0.18 0.56      0.03      0.23
```



```
In [5]: # same picture at different granularities – does the mix conclusion survive?
for freq, label in [('W', 'weekly'), ('SMS', 'semi-monthly'), ('MS', 'monthly')]:
    g = (ch.set_index('revenue_time').groupby([pd.Grouper(freq=freq), 'rev_t
        .sum().unstack(fill_value=0))
    g = g.div(g.sum(axis=1), axis=0)
    means = g.mean().round(3)
    stds = g.std().round(3)
    print(f'{label:<14} mean share: {means.to_dict()}')
    print(f'{"":14} volatility: {stds.to_dict()}')
```

```

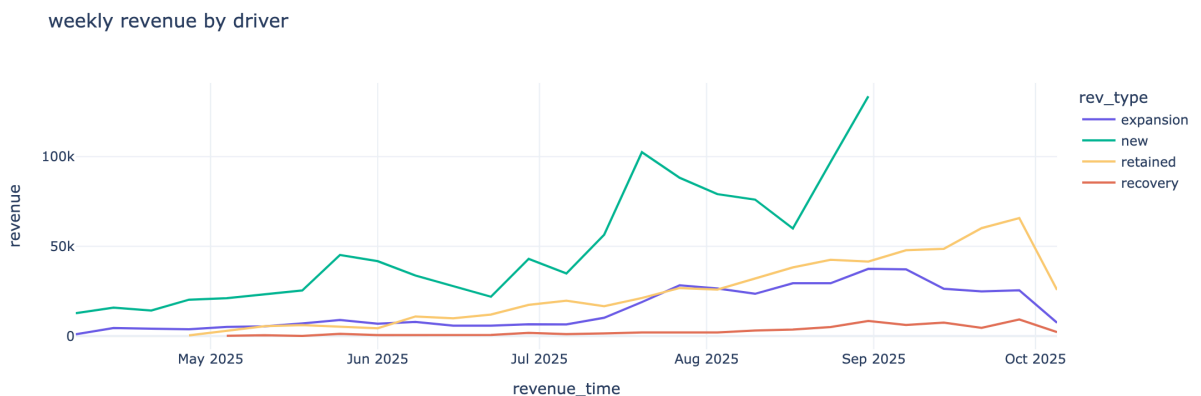
weekly = (ch.set_index('revenue_time').groupby([pd.Grouper(freq='W'), 'rev_t
        .sum().rename('revenue').reset_index())
fig = px.line(weekly, x='revenue_time', y='revenue', color='rev_type', title
fig.show()

```

```

weekly          mean share: {'expansion': 0.183, 'new': 0.542, 'recovery': 0.
026, 'retained': 0.25}
                volatility: {'expansion': 0.072, 'new': 0.281, 'recovery': 0.
026, 'retained': 0.208}
semi-monthly    mean share: {'expansion': 0.184, 'new': 0.554, 'recovery': 0.
025, 'retained': 0.237}
                volatility: {'expansion': 0.069, 'new': 0.273, 'recovery': 0.
025, 'retained': 0.197}
monthly         mean share: {'expansion': 0.183, 'new': 0.557, 'recovery': 0.
025, 'retained': 0.235}
                volatility: {'expansion': 0.063, 'new': 0.286, 'recovery': 0.
026, 'retained': 0.208}

```



```

In [6]: # ARPU = driver revenue / unique customers who contributed to that driver
# (paying customers only → strictly ARPPU; ARPU is the usual label)
cust_by_type = ch.groupby('rev_type')['customer'].nunique().rename('customers')
rev_by_type = ch.groupby('rev_type')['sales_amount'].sum().rename('revenue')
drv = pd.concat([cust_by_type, rev_by_type], axis=1)
drv['rev_share'] = drv['revenue'] / drv['revenue'].sum()
drv['arpu'] = drv['revenue'] / drv['customers']
print(drv.round(2))

fig = px.scatter(drv.reset_index(), x='customers', y='arpu', size='revenue',
                title='reach vs ARPU, by driver')
fig.update_traces(textposition='top center')
fig.update_yaxes(title='ARPU, $')
fig.show()

# same view month by month
by_m = (ch.groupby(['ym', 'rev_type'])
        .agg(customers=('customer', 'nunique'), revenue=('sales_amount', 'sum'))
        .reset_index())
by_m['arpu'] = by_m['revenue'] / by_m['customers']
print(by_m.round(2).to_string(index=False))

fig = px.scatter(by_m, x='customers', y='arpu', size='revenue', color='rev_t
                facet_col='ym', facet_col_wrap=3, text='rev_type',

```

```

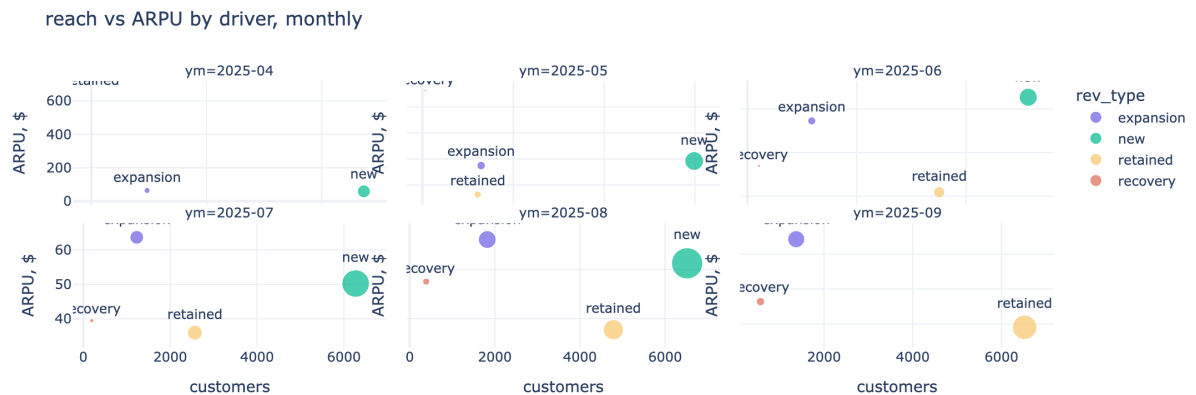
        title='reach vs ARPU by driver, monthly')
fig.update_traces(textposition='top center')
fig.update_yaxes(title='ARPU, $', matches=None)
fig.update_xaxes(matches=None)
fig.show()

```

	customers	revenue	rev_share	arpu
rev_type				
expansion	4433	405,294.89	0.19	91.43
new	19532	1,074,438.97	0.50	55.01
recovery	1183	65,477.90	0.03	55.35
retained	9098	587,999.93	0.28	64.63



ym	rev_type	customers	revenue	arpu
2025-04	expansion	242	15,509.25	64.09
2025-04	new	1183	69,446.62	58.70
2025-04	retained	2	1,324.64	662.32
2025-05	expansion	647	30,703.65	47.46
2025-05	new	2991	147,225.04	49.22
2025-05	recovery	34	2,593.35	76.27
2025-05	retained	610	22,179.02	36.36
2025-06	expansion	588	27,798.76	47.28
2025-06	new	2568	135,320.49	52.69
2025-06	recovery	106	3,919.78	36.98
2025-06	retained	1754	54,246.07	30.93
2025-07	expansion	1231	78,334.21	63.63
2025-07	new	6272	315,084.17	50.24
2025-07	recovery	196	7,739.27	39.49
2025-07	retained	2570	92,510.51	36.00
2025-08	expansion	1822	131,467.76	72.16
2025-08	new	6518	407,362.65	62.50
2025-08	recovery	386	21,247.76	55.05
2025-08	retained	4785	169,568.41	35.44
2025-09	expansion	1373	121,481.26	88.48
2025-09	recovery	568	29,977.74	52.78
2025-09	retained	6523	248,171.28	38.05



Insight

Read three things off the output above: the **share** each driver contributes, how that share **moves over time**, and how many **customers** sit behind it.

The mix is more informative than the total. New revenue dominating means the business is running on acquisition; a rising retained share means the base is starting to carry itself. Expansion's share sets the ceiling on how much an upsell programme could be worth — if it's already double-digit with no deliberate upsell surface, that's the cheapest headroom in the document.

The granularity check matters for reporting: if the weekly mix is far more volatile than the monthly one, weekly revenue-mix charts are mostly noise and monthly is the right reporting cadence. The launch spikes will show up here first.

Product conclusion. Whichever driver has a large share *and* a small customer count is where per-customer effort pays back fastest — that's usually expansion. Whichever has a large customer count and small share is where product effort pays back, because the reach is already there and the monetisation isn't.

2. Retention

Three parts: pick the right window, build features across every dimension in the data, then find what actually predicts survival.

2.1 Choosing the retention window

Most retention analyses assume a window. That assumption is usually wrong in one of two directions — too short and you measure noise, too long and you lose most of the sample to censoring. We test it instead.

```

In [7]: GRACE = 7
        PERIOD = 30

        sub = charges_df[~charges_df['is_credits']].merge(
            charges_df.groupby('customer')['revenue_time'].min().rename('first_pay')
        )
        sub['m'] = ((sub['revenue_time'] - sub['first_pay']).dt.total_seconds() // (

        base = customers_df.merge(
            charges_df.groupby('customer').agg(
                first_pay=('revenue_time', 'min'),
                last_pay=('revenue_time', 'max'),
                n_charges=('charge_id', 'size'),
                revenue=('sales_amount', 'sum'),
                first_plan=('subscription_plan', 'first'),
                first_period=('subscription_period', 'first'),
            ), left_on='customer', right_index=True, how='inner')

        base['tenure_days'] = (DATA_END_BILLING - base['first_pay']).dt.total_seconds()
        base['cohort'] = base['first_pay'].dt.tz_localize(None).dt.to_period('M').asfreq('M')

        # for each candidate window: how many customers are observable, and what is
        rows = []
        for w in [30, 45, 60, 75, 90, 120]:
            obs = base['tenure_days'] >= w + GRACE
            paid_in = sub[(sub['revenue_time'] > sub['first_pay'] + pd.Timedelta(days=w)) &&
                ((sub['revenue_time'] - sub['first_pay']).dt.total_seconds() > 0)]
            retained = base['customer'].isin(set(paid_in['customer']))
            n = obs.sum()
            r = retained[obs].mean() if n else np.nan
            rows.append({'window_days': w, 'observable': int(n),
                'observable_share': n / len(base), 'retention': r,
                'se': np.sqrt(r * (1 - r) / n) if n else np.nan})
        win = pd.DataFrame(rows)
        win['ci95'] = 1.96 * win['se']
        print(win.round(4))

```

	window_days	observable	observable_share	retention	se	ci95
0	30	18048	0.92	0.47	0.00	0.01
1	45	15268	0.78	0.50	0.00	0.01
2	60	11994	0.61	0.54	0.00	0.01
3	75	8067	0.41	0.56	0.01	0.01
4	90	6127	0.31	0.57	0.01	0.01
5	120	3600	0.18	0.60	0.01	0.02

```

In [8]: fig = make_subplots(specs=[[{'secondary_y': True}]]
        fig.add_bar(x=win['window_days'], y=win['observable'], name='observable cust',
            marker_color=PALETTE[6])
        fig.add_scatter(x=win['window_days'], y=win['retention'], name='retention',
            error_y=dict(type='data', array=win['ci95']),
            line=dict(color=PALETTE[0], width=3), secondary_y=True)
        fig.update_yaxes(title='customers with enough history')
        fig.update_yaxes(title='retention', tickformat='.0%', secondary_y=True)
        fig.update_xaxes(title='window, days', tickmode='array', tickvals=win['window_days'],
            ticktext=[f'{d}d' for d in win['window_days']])
        fig.update_layout(title='the retention window trade-off: sample size vs hori

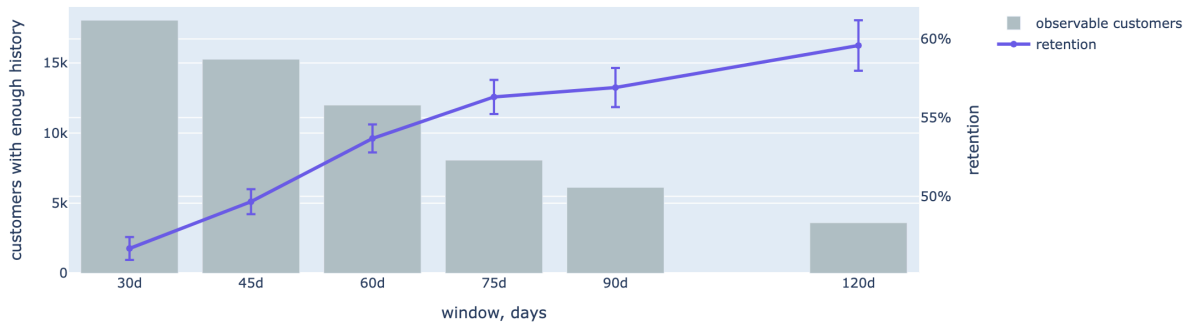
```



```
fig.show()
```

```
print(f'sample lost going 30d -> 90d: {1 - win.loc[win["window_days"]==90, "
```

the retention window trade-off: sample size vs horizon



sample lost going 30d -> 90d: 66.1%

```
In [9]: # where does the survival curve actually bend? that's the window that carries
alive = sub.groupby(['customer', 'm']).size().reset_index(name='n')
mon = base[base['first_period'] == 'month'].copy()
mon['opps'] = np.floor((mon['tenure_days'] - GRACE) / PERIOD).clip(lower=0)

# each period k uses a DIFFERENT eligible set (opps >= k), not one fixed cohort
# so S(t) can rise when later periods are estimated on earlier (more durable) data
surv = []
for k in range(0, 6):
    elig = mon[mon['opps'] >= k]
    if len(elig) < 30:
        continue
    ok = set(alive.loc[alive['m'] == k, 'customer'])
    surv.append({'period': k + 1, 'n_eligible': len(elig),
                'alive': elig['customer'].isin(ok).sum(),
                'survival': elig['customer'].isin(ok).mean()})
surv = pd.DataFrame(surv)
surv['step_drop'] = surv['survival'].shift(1) - surv['survival']
print(surv.round(4))

fig = make_subplots(rows=1, cols=2, subplot_titles=('survival S(t)', 'drop in survival'))
fig.add_scatter(x=surv['period'], y=surv['survival'], mode='lines+markers',
                line=dict(color=PALETTE[0], width=3), row=1, col=1)
fig.add_bar(x=surv['period'], y=surv['step_drop'], marker_color=PALETTE[3],
            row=1, col=2)
fig.update_xaxes(title='period', tickmode='array', tickvals=list(range(1, 7)),
                range=[0.5, 6.5], row=1, col=1)
fig.update_xaxes(title='period', tickmode='array', tickvals=list(range(1, 7)),
                range=[0.5, 6.5], row=1, col=2)
fig.update_layout(title='where the curve bends', showlegend=False)
fig.show()

# funnel on headcounts (same caveat: denominators shrink with tenure)
funnel = surv[['period', 'alive', 'n_eligible', 'survival']].copy()
funnel['stage'] = [f'period {p}' for p in funnel['period']]
funnel['label'] = funnel.apply(
    lambda r: f'{int(r.alive):,} ({r.survival:.0%} of {int(r.n_eligible):,})',
    axis=1)
fig = px.funnel(funnel, x='alive', y='stage', text='label',
```

```

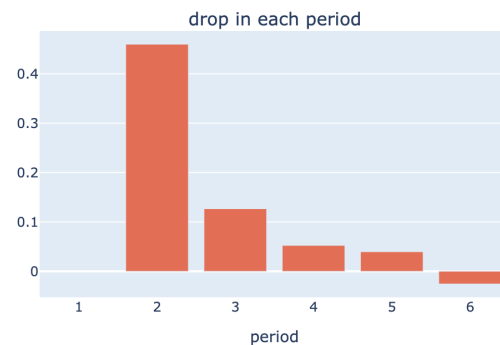
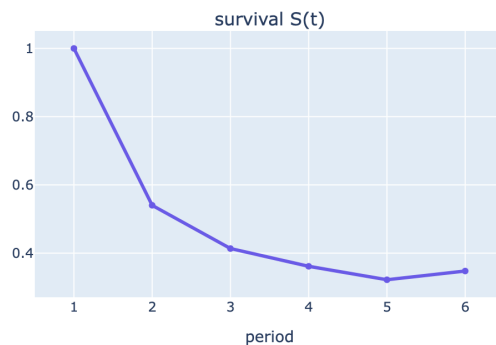
        title='retention funnel: customers still paying at each period'
fig.update_traces(textposition='inside', textinfo='text')
fig.show()

if surv['step_drop'].notna().any():
    worst = surv.loc[surv['step_drop'].idxmax()]
    print(f'\nbiggest single drop at period {int(worst["period"])}: {worst["step_drop"]}')
    pos = surv['step_drop'].clip(lower=0)
    if len(surv) > 1 and pos.sum() > 0:
        print(f'share of all (positive) churn happening in period 2: '
              f'{pos.iloc[1] / pos.sum():.1%}')
    if (surv['step_drop'] < 0).any():
        print('note: negative step_drop means S(t) rose – eligible sample shrinks to earlier cohorts only, so rates are not comparable to a fixed-cohort KM curve')

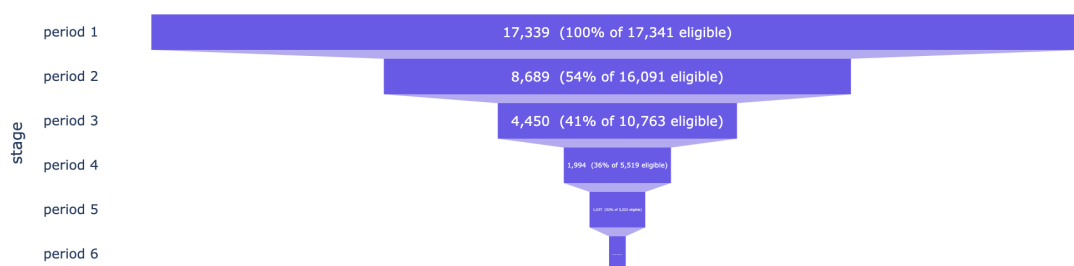
```

	period	n_eligible	alive	survival	step_drop
0	1	17341	17339	1.00	NaN
1	2	16091	8689	0.54	0.46
2	3	10763	4450	0.41	0.13
3	4	5519	1994	0.36	0.05
4	5	3222	1037	0.32	0.04
5	6	892	310	0.35	-0.03

where the curve bends



retention funnel: customers still paying at each period



biggest single drop at period 2: 46.0%

share of all (positive) churn happening in period 2: 67.8%

note: negative step_drop means S(t) rose – eligible sample shrinks to earlier cohorts only, so rates are not comparable to a fixed-cohort KM curve

```

In [10]: # retention structure by cohort – is the level moving, the shape, or both?
coh_surv = []

```

```

for c, d in mon.groupby('cohort'):
    if len(d) < 50:
        continue
    for k in range(0, 5):
        elig = d[d['ops'] >= k]
        if len(elig) < 30:
            continue
        ok = set(alive.loc[alive['m'] == k, 'customer'])
        coh_surv.append({'cohort': c, 'period': k, 'n': len(elig),
                        'survival': elig['customer'].isin(ok).mean()})
coh_surv = pd.DataFrame(coh_surv)

tab = coh_surv.pivot(index='cohort', columns='period', values='survival')
print(f'survival by cohort:\n{tab.round(3)}')

m1 = coh_surv[coh_surv['period'] == 1].set_index('cohort')
print(f'\nM1 retention spread: {m1["survival"].min():.1%} to {m1["survival"]
      f'({m1["survival"].max() - m1["survival"].min():.1%} pp)')

fig = px.imshow(tab, text_auto='.0%', aspect='auto', color_continuous_scale=
                title='survival by cohort and billing period')
fig.show()

fig = px.line(coh_surv, x='period', y='survival', color='cohort', markers=True,
              title='retention curves by cohort')
fig.update_yaxes(tickformat='.0%')
fig.show()

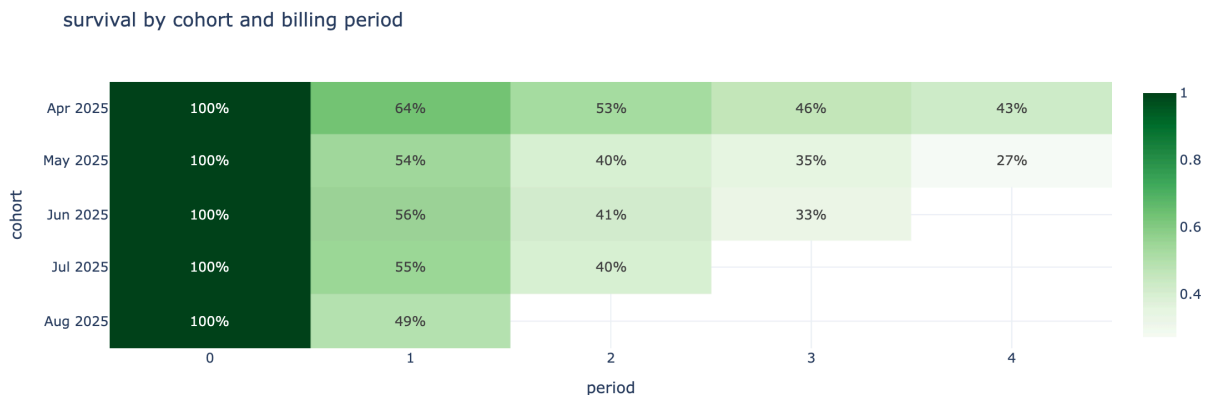
```

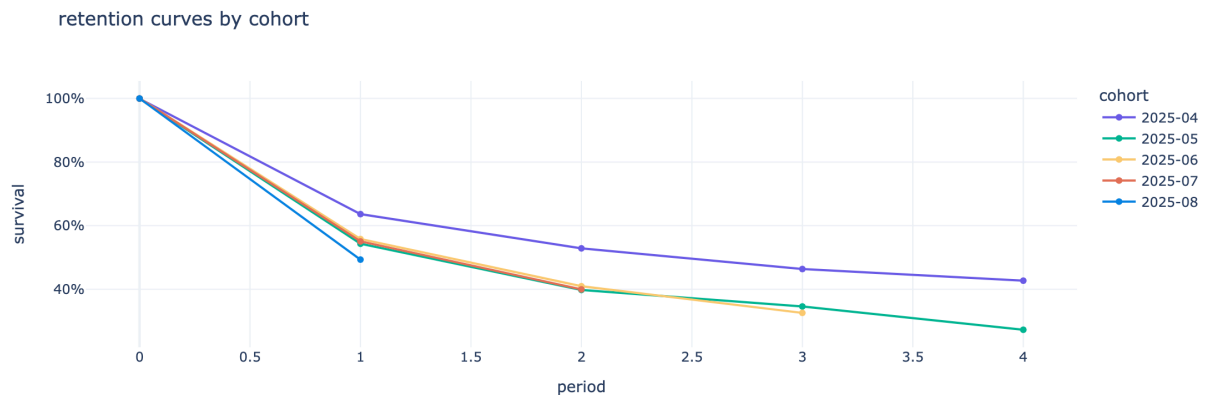
```

survival by cohort:
period    0    1    2    3    4
cohort
2025-04  1.00  0.64  0.53  0.46  0.43
2025-05  1.00  0.54  0.40  0.35  0.27
2025-06  1.00  0.56  0.41  0.33  NaN
2025-07  1.00  0.55  0.40  NaN  NaN
2025-08  1.00  0.49  NaN  NaN  NaN

```

M1 retention spread: 49.3% to 63.6% (14.3% pp)





```
In [11]: # is the DIFFERENCE between cohorts a level shift or a shape change?
# level shift -> same product, different customer quality
# shape change -> the product itself worked differently for them
norm = coh_surv.merge(m1['survival'].rename('m1'), on='cohort')
norm['rel'] = norm['survival'] / norm['m1']

shape = norm[norm['period'] >= 1].pivot(index='cohort', columns='period', va
print(f'survival rescaled to each cohort\'s own M1:\n{shape.round(3)}')

if shape.shape[1] > 1:
    late = shape.columns[-1]
    print(f'\nspread at M1 (raw levels):      {m1["survival"].max() - m1["sur
    print(f'spread at M{late} after rescaling: {shape[late].max() - shape[la
    print(f'-> {"LEVEL shift: cohorts differ at M1 then decay alike" if (sha

fig = px.line(norm[norm['period'] >= 1], x='period', y='rel', color='cohort'
            title='survival rescaled to each cohort M1 - do the shapes ove
fig.show()
```

survival rescaled to each cohort's own M1:

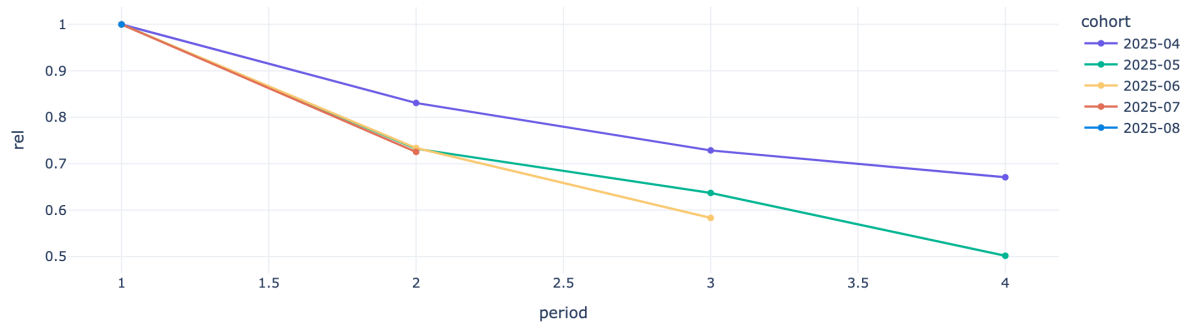
period	1	2	3	4
cohort				
2025-04	1.00	0.83	0.73	0.67
2025-05	1.00	0.73	0.64	0.50
2025-06	1.00	0.73	0.58	NaN
2025-07	1.00	0.72	NaN	NaN
2025-08	1.00	NaN	NaN	NaN

spread at M1 (raw levels): 14.3% pp

spread at M4 after rescaling: 16.9% pp

-> SHAPE change: cohorts decay differently, not just from different starting points

survival rescaled to each cohort M1 - do the shapes overlap?



Insight — the window is a choice with a cost, and the data can pick it

Three findings decide it:

1. **The sample cost.** Moving from a 30-day to a 90-day window drops a large share of customers to censoring, because most of the base signed up recently. Wider windows aren't free.
2. **Where the churn is.** If period 0→1 accounts for most of the total drop, a 30-day window captures nearly all the signal and the longer windows mostly add variance.
3. **Precision.** The confidence interval widens as the sample shrinks, so a 90-day retention estimate may be *less* certain than a 30-day one despite covering more of the customer's life.

Decision for this analysis: the first renewal (30 days + grace) is the modelling target. It has the most sample, the most churn, and is early enough to act on. Longer windows are reported alongside as a check, not as the target.

Product conclusion. This also settles the dashboard question — a single "first-renewal rate by cohort" tile is a better health metric than 90-day retention, which arrives too late to change anything and is measured on a shrinking sample.